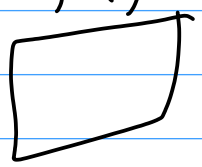
int main() {

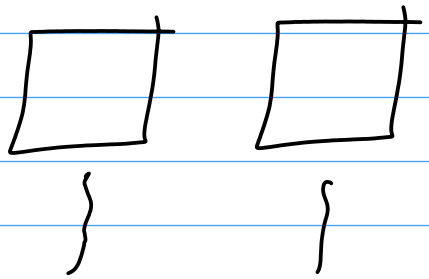
{

}

single process



fork, thread



var var

**3.1** Using the program shown in Figure 3.30, explain what the output will be at LINE A.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int value = 5;

int main()
{
pid_t pid;

  pid = fork();

  if (pid == 0) { /* child process */
     value += 15;
     return 0;
  }
  else if (pid > 0) { /* parent process */
     wait(NULL);
     printf("PARENT: value = %d",value); /* LINE A */
     return 0;
  }
}
```

Par

Val = 5
Pid = 1

Child

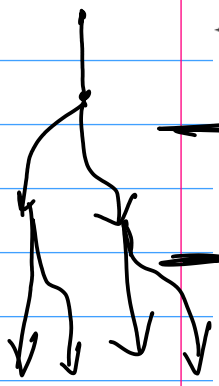
val =
pid = 0

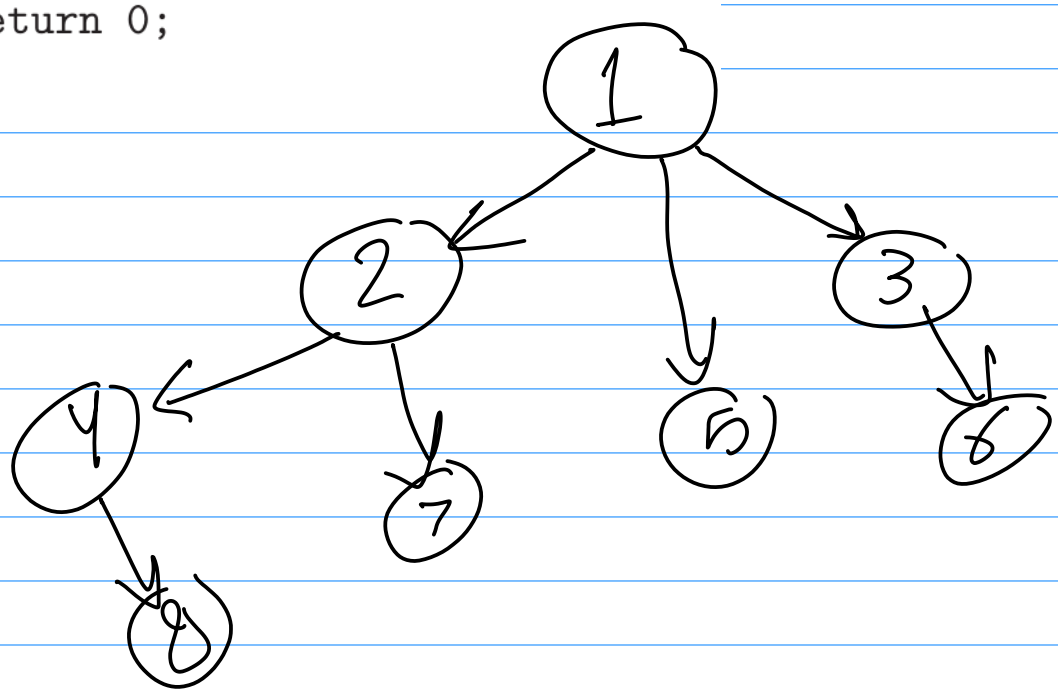
20

5

**3.2** Including the initial parent process, how many processes are created by the program shown in Figure 3.31?

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    /* fork a child process */
    fork();

    /* fork another child process */
    fork();

    /* and fork another */
    fork();

    return 0;
}
```

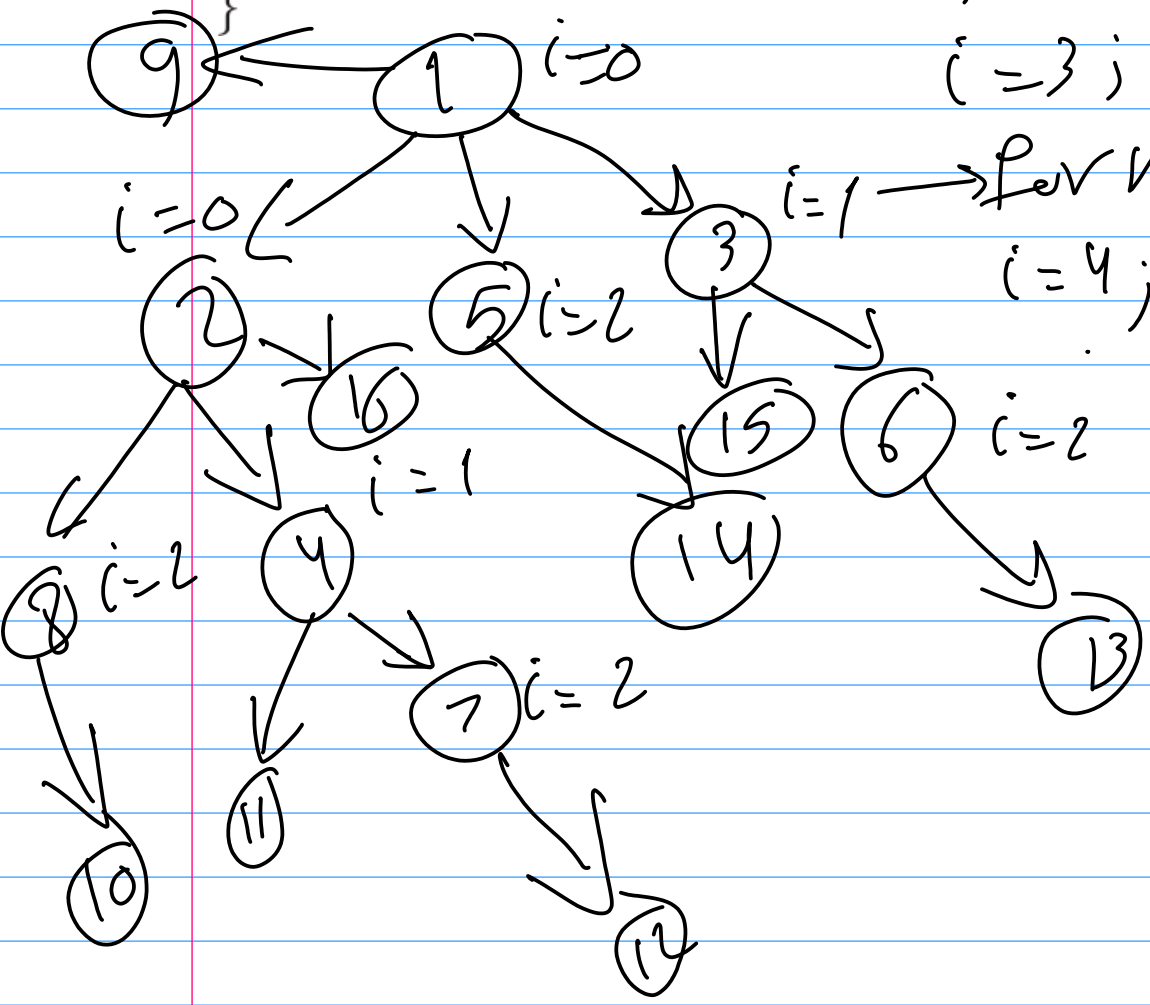




**3.12** Including the initial parent process, how many processes are created by the program shown in Figure 3.32?

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    int i;   j

    for (i = 0; i < 4; i++)
        fork();

    return 0;
}
```

i=0
fork();
i=1;
fork();
i=2;
fork();
i=3;
fork();
i=4;



**3.13** Explain the circumstances under which which the line of code marked `printf("LINE J")` in Figure 3.33 will be reached.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
      fprintf(stderr, "Fork Failed");
      return 1;
    }
    else if (pid == 0) { /* child process */
      execlp("/bin/ls","ls",NULL);
      printf("LINE J");
    }
    else { /* parent process */
      /* parent will wait for the child to complete */
      wait(NULL);
      printf("Child Complete");
    }

    return 0;
}
```

stdout ✓

stderr ✓

child && fail exec

par

child

LS

exec

**3.14** Using the program in Figure 3.34, identify the values of pid at lines A, B, C, and D. (Assume that the actual pids of the parent and child are 2600 and 2603, respectively.)

| pid par 2600 | Child 2603 |
|---|---|

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid, pid1;

   /* fork a child process */
   pid = fork();

   if (pid < 0) { /* error occurred */
      fprintf(stderr, "Fork Failed");
      return 1;
   }
   else if (pid == 0) { /* child process */
      pid1 = getpid();
      printf("child: pid = %d",pid); /* A */
      printf("child: pid1 = %d",pid1); /* B */
   }
   else { /* parent process */
      pid1 = getpid();
      printf("parent: pid = %d",pid); /* C */
      printf("parent: pid1 = %d",pid1); /* D */
      wait(NULL);
   }

   return 0;
}
```

| child | Par |
|---|---|
| pid = 0 | pid = 2603 |
| pid1 = 2603 | pid1 = 2600 |

A: 0

B: 2603

C: 2603

D: 2600

**3.17** Using the program shown in Figure 3.35, explain what the output will be at lines X and Y.

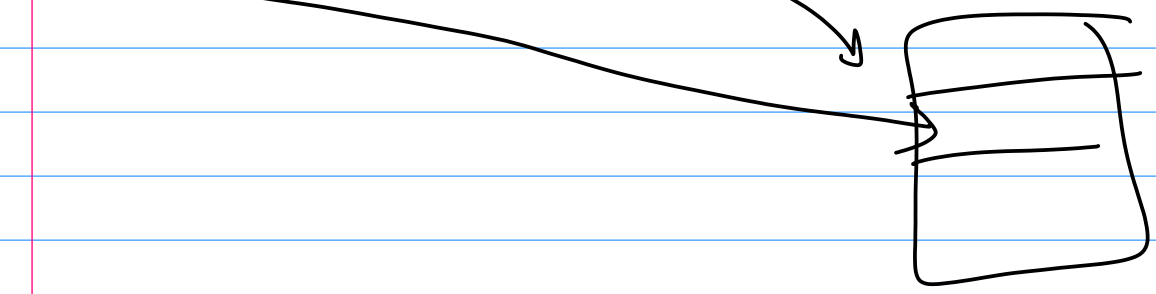
```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

#define SIZE 5

int nums[SIZE] = {0,1,2,3,4};

int main()
{
int i;
pid_t pid;

   pid = fork();

   if (pid == 0) {
      for (i = 0; i < SIZE; i++) {
         nums[i] *= -i;
         printf("CHILD: %d ",nums[i]); /* LINE X */
      }
   }
   else if (pid > 0) {
      wait(NULL);
      for (i = 0; i < SIZE; i++)
         printf("PARENT: %d ",nums[i]); /* LINE Y */
   }

   return 0;
}
```

nums = {0, -1, -4, -9, -16}

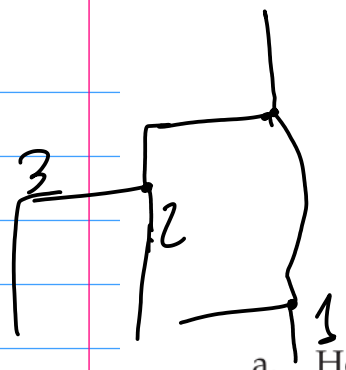
0, 1, 2, 3, 4

ptr = 0xff...

ptr



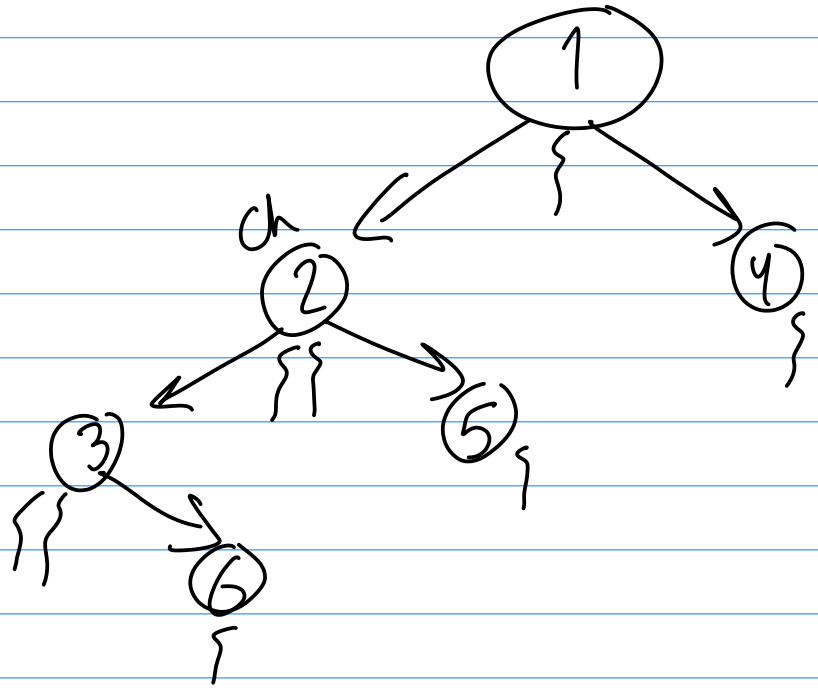
**4.15** Consider the following code segment:

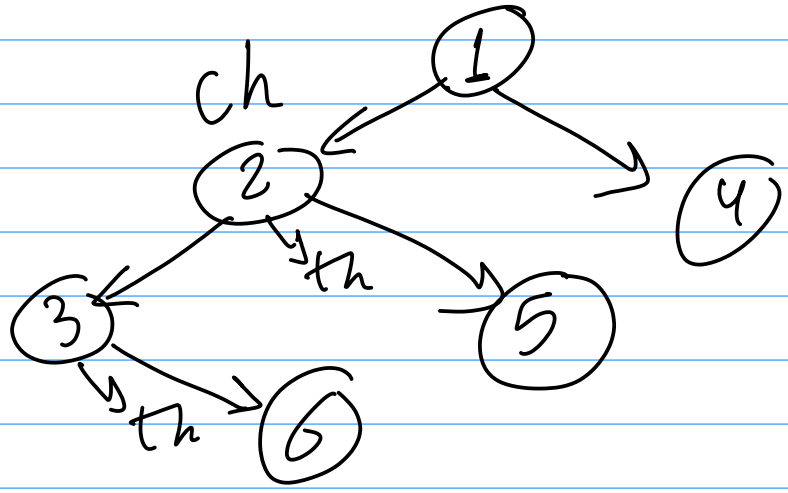


```
pid_t pid;

pid = fork();
if (pid == 0) { /* child process */
   fork();
   thread_create( . . .);
}
fork();
```

a. How many unique processes are created? 6

b. How many unique threads are created? 6+2=8



**4.17** The program shown in Figure 4.16 uses the Pthreads API. What would be the output from the program at LINE C and LINE P?

```
#include <pthread.h>
#include <stdio.h>

#include <types.h>

int value = 0;
void *runner(void *param); /* the thread */

int main(int argc, char *argv[])
{
pid_t pid;
pthread_t tid;
pthread_attr_t attr;

   pid = fork();

   if (pid == 0) { /* child process */
      pthread_attr_init(&attr);
      pthread_create(&tid,&attr,runner,NULL);
      pthread_join(tid,NULL);
      printf("CHILD: value = %d",value); /* LINE C */
   }
   else if (pid > 0) { /* parent process */
      wait(NULL);
      printf("PARENT: value = %d",value); /* LINE P */
   }
}

void *runner(void *param) {
   value = 5;
   pthread_exit(0);
}
```

Par

V=0

ch

V=5

thread

5

$ ls /bin

int main(

int argc

char *argv[])

"ls" "/bin"